Intelligent Agents

Russel Norvig, *Artificial Intelligence: a modern approach*
Chapter 2: Intelligent agents

Giacomo Fiumara (gfiumara@unime.it)

Outline

Agents and Environments

Good Behavior: The Concept of Rationality

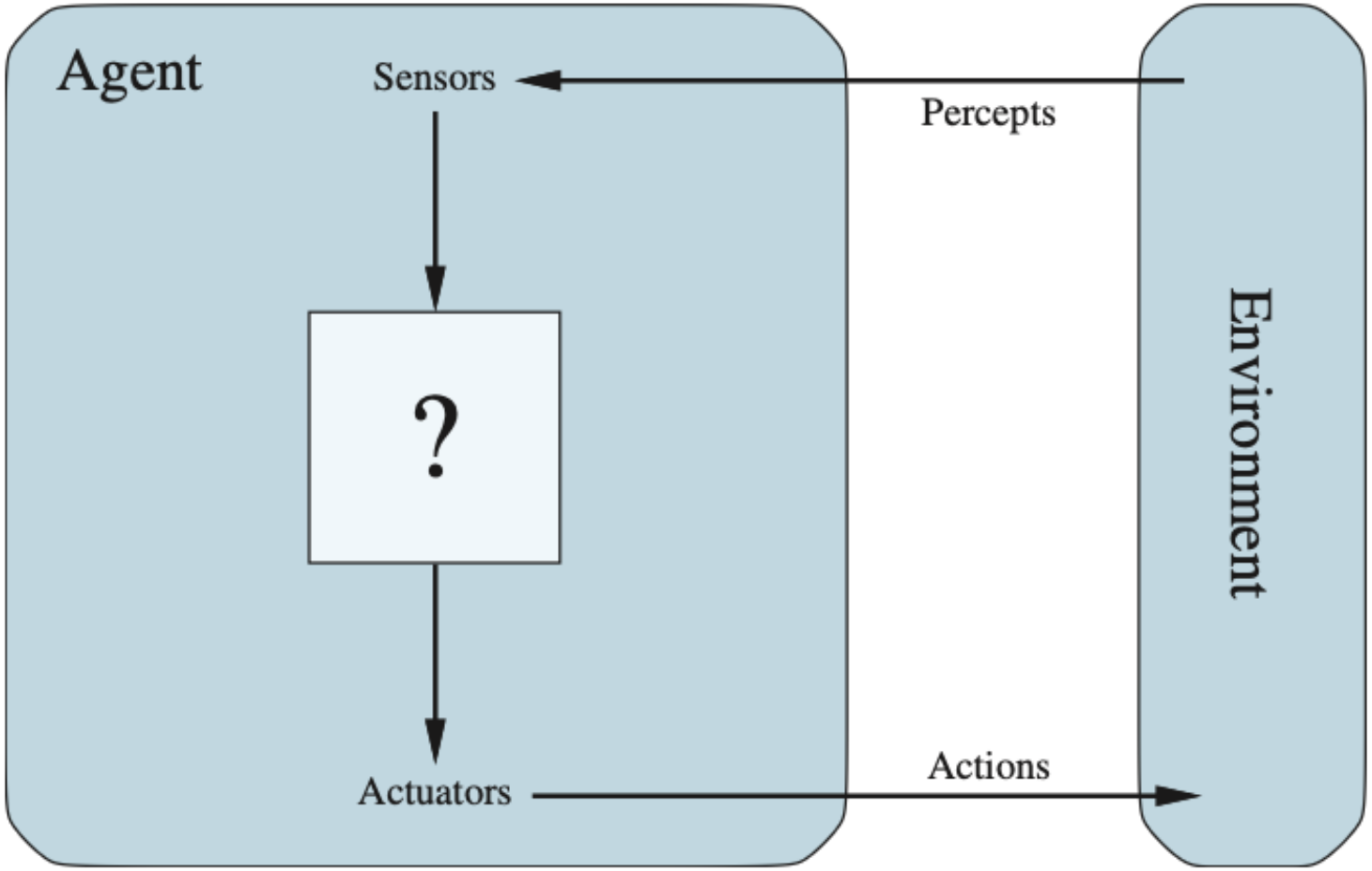
The Nature of Environments

The Structure of Agents

Agents and Environments

What is an Agent?

- An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.
- A human agent has eyes, ears, and other organs for sensors and hands, legs, vocal tract, and so on for actuators.
- A robotic agent might have cameras and infrared range finders for sensors and various motors for actuators.
- A software agent receives file contents, network packets, and human input (keyboard/mouse/touchscreen/voice) as sensory inputs and acts on the environment by writing files, sending network packets, and displaying information.

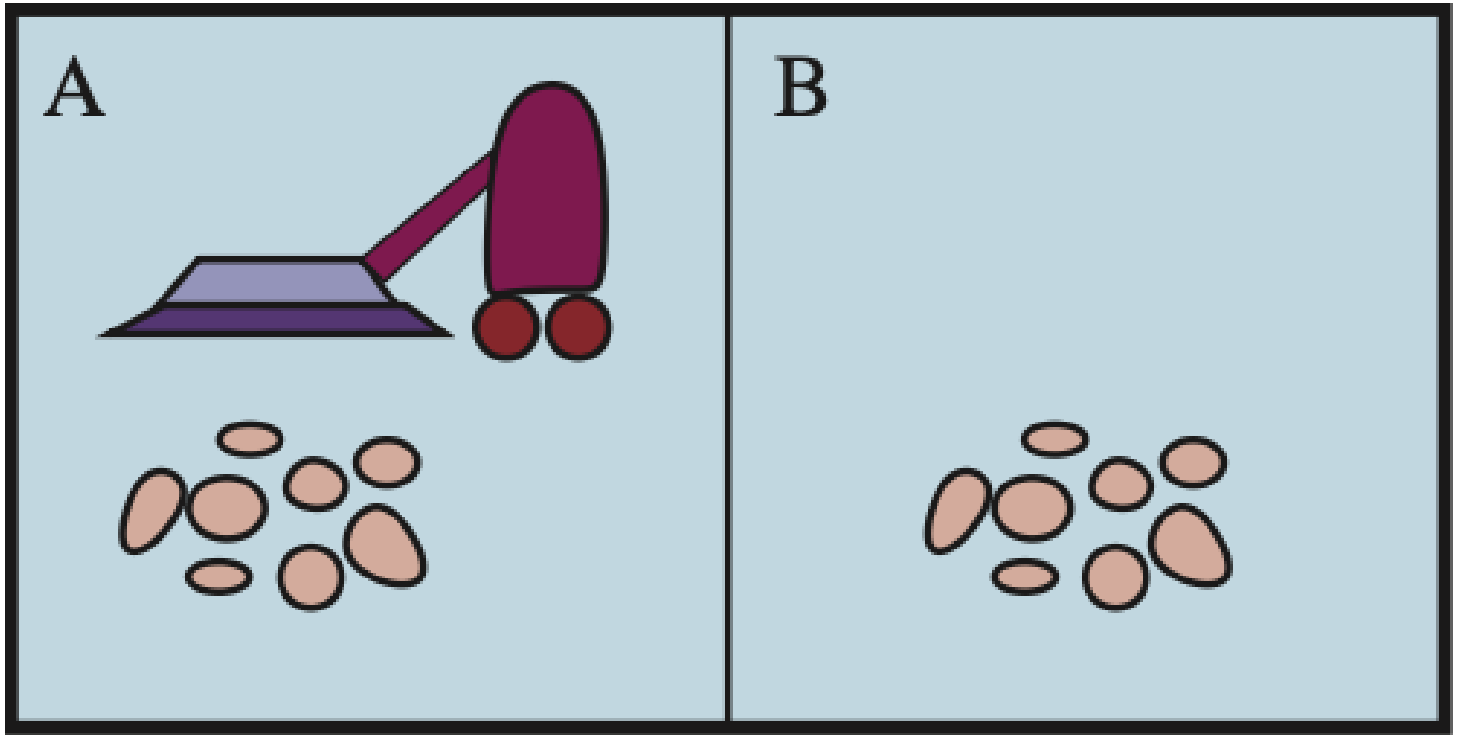

Agent
Sensors
Percepts
?
Environment
Actuators
Actions

Percepts and Percept Sequences

- A percept is the content that an agent's sensors are perceiving at a given instant.
- A percept sequence is the complete history of everything the agent has ever perceived.
- An agent's choice of action at any given instant can depend on its built-in knowledge and on the entire percept sequence observed to date, but not on anything it hasn't perceived.
- An agent function is a mathematical description that maps any given percept sequence to an action.
- An agent program is a concrete implementation that runs within some physical system.

The Vacuum-Cleaner World

- A robotic vacuum-cleaning agent operates in a world consisting of squares that can be either dirty or clean.
- The vacuum agent perceives which square it is in and whether there is dirt in the square.
- The available actions are: move right, move left, suck up the dirt, or do nothing.
- One simple agent function: if the current square is dirty, then suck; otherwise, move to the other square.
- The key question: What is the right way to fill out the agent function? What makes an agent good or bad, intelligent or stupid?


A
B

Percept sequence	Action
$[A, Clean]$	*Right*
$[A, Dirty]$	*Suck*
$[B, Clean]$	*Left*
$[B, Dirty]$	*Suck*
$[A, Clean], [A, Clean]$	*Right*
$[A, Clean], [A, Dirty]$	*Suck*
$\vdots$	$\vdots$
$[A, Clean], [A, Clean], [A, Clean]$	*Right*
$[A, Clean], [A, Clean], [A, Dirty]$	*Suck*
$\vdots$	$\vdots$

Good Behavior: The Concept of Rationality

What Makes an Agent Rational?

- A rational agent is one that does the right thing.
- AI generally adopts consequentialism: we evaluate an agent's behavior by its consequences.
- When an agent is placed in an environment, it generates a sequence of actions according to the percepts it receives.
- This sequence of actions causes the environment to go through a sequence of states.
- If the sequence of environment states is desirable, then the agent has performed well.

Performance Measures 1/2

- A performance measure evaluates any given sequence of environment states.
- The notion of desirability is captured by the performance measure.
- For machines, the performance measure is initially in the mind of the designer or the users the machine is designed for.

Performance Measures 2/2

- Key principle: It is better to design performance measures according to what one actually wants to be achieved in the environment, rather than according to how one thinks the agent should behave.

- Example: A vacuum-cleaner agent should be rewarded for having a clean floor, not for the amount of dirt it cleaned (which could lead to dumping and re-cleaning).

Definition of Rational Agent

- Rationality depends on four things:
  - The performance measure that defines the criterion of success.
  - The agent's prior knowledge of the environment.
  - The actions that the agent can perform.
  - The agent's percept sequence to date.

- A rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

- Rationality depends on the task environment; the same agent can be rational in one environment but irrational in another.

Rationality vs. Omniscience

- Rationality is not the same as perfection.
- An omniscient agent knows the actual outcome of its actions and can act accordingly; but omniscience is impossible in reality.
- Rationality maximizes expected performance, while perfection maximizes actual performance.
- A rational agent does not require omniscience because the rational choice depends only on the percept sequence to date.
- A rational agent should choose the "looking" action before stepping into the street, because looking helps maximize expected performance. This is called information gathering.

Learning and Autonomy 1/2

- A rational agent must not only gather information but also learn as much as possible from what it perceives.
- To the extent that an agent relies on the prior knowledge of its designer rather than on its own percepts and learning processes, the agent lacks autonomy.
- A rational agent should be autonomous—it should learn what it can to compensate for partial or incorrect prior knowledge.

Learning and Autonomy 2/2

- One need not require complete autonomy from the start. Just as evolution provides animals with built-in reflexes to survive long enough to learn, artificial agents can be provided with some initial knowledge plus an ability to learn.

- After sufficient experience, the behavior of a rational agent can become effectively independent of its prior knowledge.

The Nature of Environments

Task Environments and PEAS

- A task environment specifies the performance measure, the external environment, the actuators, and the sensors.
- PEAS stands for: Performance measure, Environment, Actuators, and Sensors.
- In designing an agent, the first step must always be to specify the task environment as fully as possible.
- The nature of the task environment directly affects the appropriate design for the agent program.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments	Touchscreen/voice entry of symptoms and findings
Satellite image analysis system	Correct categorization of objects, terrain	Orbiting satellite, downlink, weather	Display of scene categorization	High-resolution digital camera
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, tactile and joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, raw materials, operators	Valves, pumps, heaters, stirrers, displays	Temperature, pressure, flow, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, feedback, speech	Keyboard entry, voice

Observable vs. Partially Observable 1/2

- If an agent's sensors give it access to the complete state of the environment at each point in time, the task environment is fully observable.
- A task environment is effectively fully observable if the sensors detect all aspects relevant to the choice of action.
- Fully observable environments are convenient because the agent need not maintain any internal state to keep track of the world.

Observable vs. Partially Observable 2/2

- An environment is partially observable if sensors are noisy or inaccurate, or if parts of the state are simply missing from the sensor data.
- If the agent has no sensors at all, then the environment is unobservable. Yet, the agent's goals may still be achievable, sometimes with certainty.

Single-Agent vs. Multiagent 1/2

- In a single-agent environment, there is only one agent whose behavior matters.
- In a multiagent environment, multiple agents interact and their behaviors affect each other.
- An object B should be treated as an agent if its behavior is best described as maximizing a performance measure whose value depends on agent A's behavior.

Single-Agent vs. Multiagent 2/2

- In chess, the opponent is trying to maximize its performance measure, which minimizes agent A's performance measure. Thus, chess is a competitive multiagent environment.

- In taxi driving, avoiding collisions maximizes the performance measure of all agents, so it is a partially cooperative multiagent environment. It is also partially competitive (only one car can occupy a parking space).

Deterministic vs. Nondeterministic

- If the next state of the environment is completely determined by the current state and the action executed by the agent(s), the environment is deterministic.
- Otherwise, it is nondeterministic.
- In principle, an agent need not worry about uncertainty in a fully observable, deterministic environment.
- Most real situations are so complex that it is impossible to keep track of all unobserved aspects. For practical purposes, they must be treated as nondeterministic.
- Stochastic denotes a model that explicitly deals with probabilities; nondeterministic denotes possibilities listed without being quantified.

Episodic vs. Sequential 1/2

- In an episodic task environment, the agent's experience is divided into atomic episodes.
- In each episode, the agent receives a percept and then performs a single action.
- Crucially, the next episode does not depend on the actions taken in previous episodes.
- Many classification tasks are episodic. For example, an agent spotting defective parts on an assembly line bases each decision on the current part.

Episodic vs. Sequential 2/2

- In sequential environments, the current decision could affect all future decisions. Chess and taxi driving are sequential.
- Episodic environments are much simpler than sequential environments because the agent does not need to think ahead.

Static vs. Dynamic

- If the environment can change while an agent is deliberating, the environment is dynamic for that agent.
- Otherwise, it is static.
- Static environments are easy to deal with because the agent need not keep looking at the world while deciding on an action, nor need it worry about the passage of time.
- Dynamic environments continuously ask the agent what it wants to do; if it hasn't decided yet, that counts as deciding to do nothing.
- If the environment itself does not change with the passage of time but the agent's performance score does, the environment is semidynamic. Chess with a clock is semidynamic.

Discrete vs. Continuous

- The discrete/continuous distinction applies to the state of the environment, to the way time is handled, and to the percepts and actions of the agent.
- The chess environment has a finite number of distinct states (excluding the clock). Chess also has a discrete set of percepts and actions.
- Taxi driving is a continuous-state and continuous-time problem: the speed and location of the taxi and of other vehicles sweep through a range of continuous values.
- Taxi-driving actions are also continuous (steering angles, etc.).
- Input from digital cameras is discrete, but is typically treated as representing continuously varying intensities and locations.

Known vs. Unknown Environments

- In a known environment, the outcomes (or outcome probabilities if nondeterministic) for all actions are given.
- In an unknown environment, the agent will have to learn how it works in order to make good decisions.
- The distinction between known and unknown environments is not the same as the one between fully and partially observable environments.
- It is possible for a known environment to be partially observable (e.g., solitaire card games).
- Conversely, an unknown environment can be fully observable (e.g., a new video game where the screen shows the entire game state).

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

The Structure of Agents

Agent Architecture and Program

- The job of AI is to design an agent program that implements the agent function—the mapping from percepts to actions.
- An agent program runs on some sort of computing device with physical sensors and actuators, called the agent architecture.
- The relationship is: agent = architecture + program.
- The program we choose must be appropriate for the architecture.
- The architecture makes the percepts from the sensors available to the program, runs the program, and feeds the program's action choices to the actuators.

Table-Driven Agents 1/2

- One approach is to use a table-driven agent that keeps track of the percept sequence and uses it to index into a table of actions.
- The table represents explicitly the agent function that the agent program embodies.
- To build a rational agent in this way, we as designers must construct a table with the appropriate action for every possible percept sequence.

Table-Driven Agents 2/2

- However, this approach is doomed to failure due to the exponential growth of the lookup table.
- The key challenge for AI is to find out how to write programs that produce rational behavior from a smallish program rather than from a vast table.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
              table, a table of actions, indexed by percept sequences, initially fully specified

  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

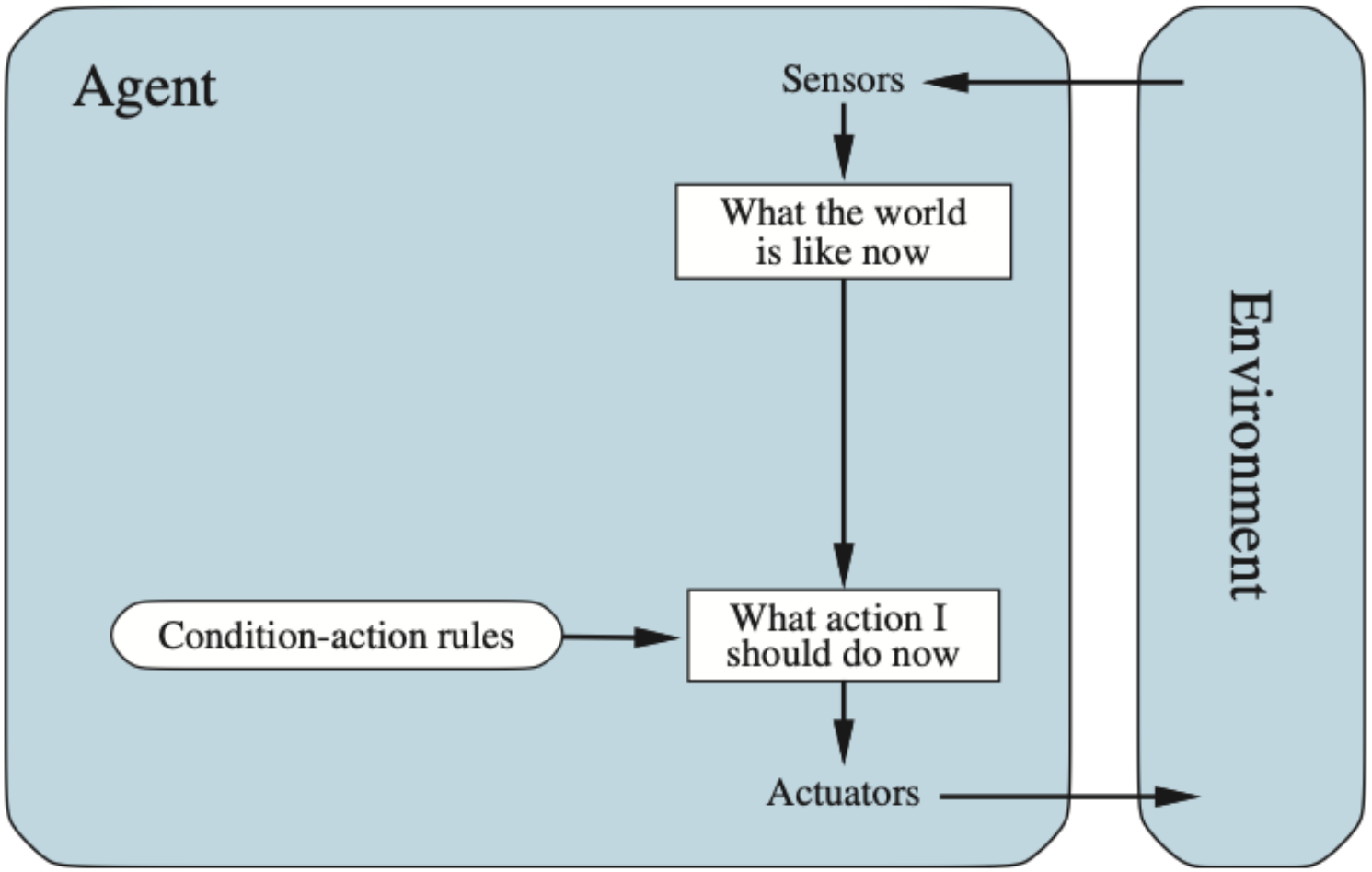
Four Basic Agent Types

- Simple reflex agents: Select actions on the basis of the current percept, ignoring the rest of the percept history.
- Model-based reflex agents: Keep track of the part of the world the agent can't see by maintaining internal state.
- Goal-based agents: Act so as to achieve their goals; they require knowledge about the future consequences of their actions.
- Utility-based agents: Select actions to maximize utility; they can handle conflicting goals and multiple achievable goals with uncertainty.

Simple Reflex Agents

- The simplest kind of agent is the simple reflex agent.
- These agents select actions on the basis of the current percept, ignoring the rest of the percept history.
- The vacuum agent whose decision is based only on the current location and on whether that location contains dirt is a simple reflex agent.
- Simple reflex agents use condition-action rules, written as: if car-in-front-is-braking then initiate-braking.
- Simple reflex agents have the admirable property of being simple, but they are of limited intelligence and work only if the correct decision can be made on the basis of just the current percept.

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```



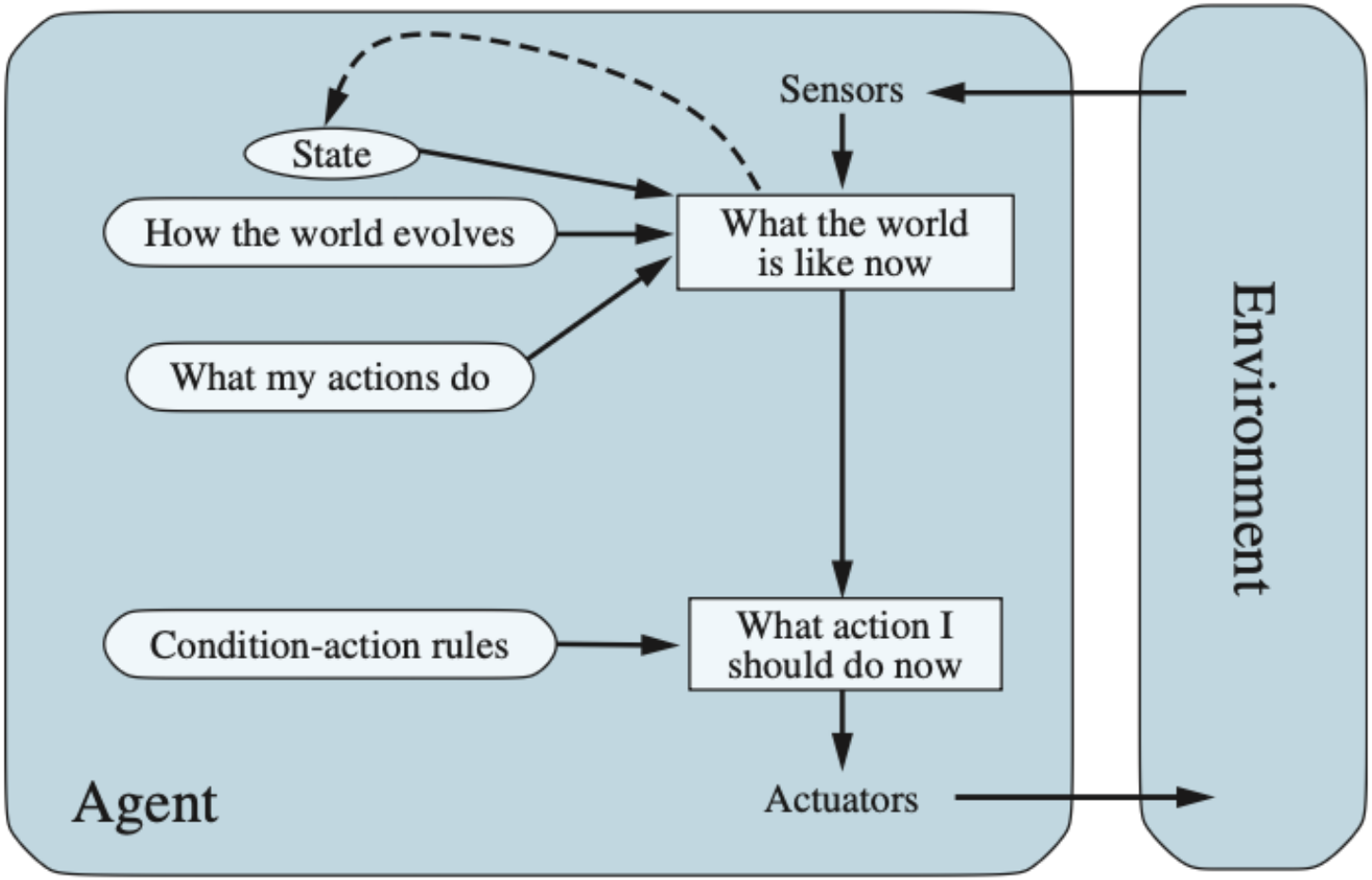
```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Limitations of Simple Reflex Agents

- A little bit of unobservability can cause serious trouble.
- Even when an agent has multiple sensors, if some conditions cannot be determined from a single percept, the agent can get stuck in infinite loops.
- For example, a simple reflex vacuum agent without a location sensor, having only a dirt sensor, cannot decide whether to move right or left when the current square is clean.
- Escape from infinite loops is possible if the agent can randomize its actions.
- However, in most cases we can do much better with more sophisticated deterministic agents.

Model-Based Reflex Agents

- The most effective way to handle partial observability is for the agent to keep track of the part of the world it can't see now.
- The agent maintains some sort of internal state that depends on the percept history and reflects at least some of the unobserved aspects of the current state.
- Updating this internal state requires two kinds of knowledge:
  - Transition model: Information about how the world changes over time (effects of the agent's actions and how the world evolves independently).
  - Sensor model: Information about how the state of the world is reflected in the agent's percepts.
- An agent that uses such models is called a model-based agent.


Sensors
State
How the world evolves
What the world is like now
What my actions do
Condition-action rules
What action I should do now
Actuators
Agent
Environment

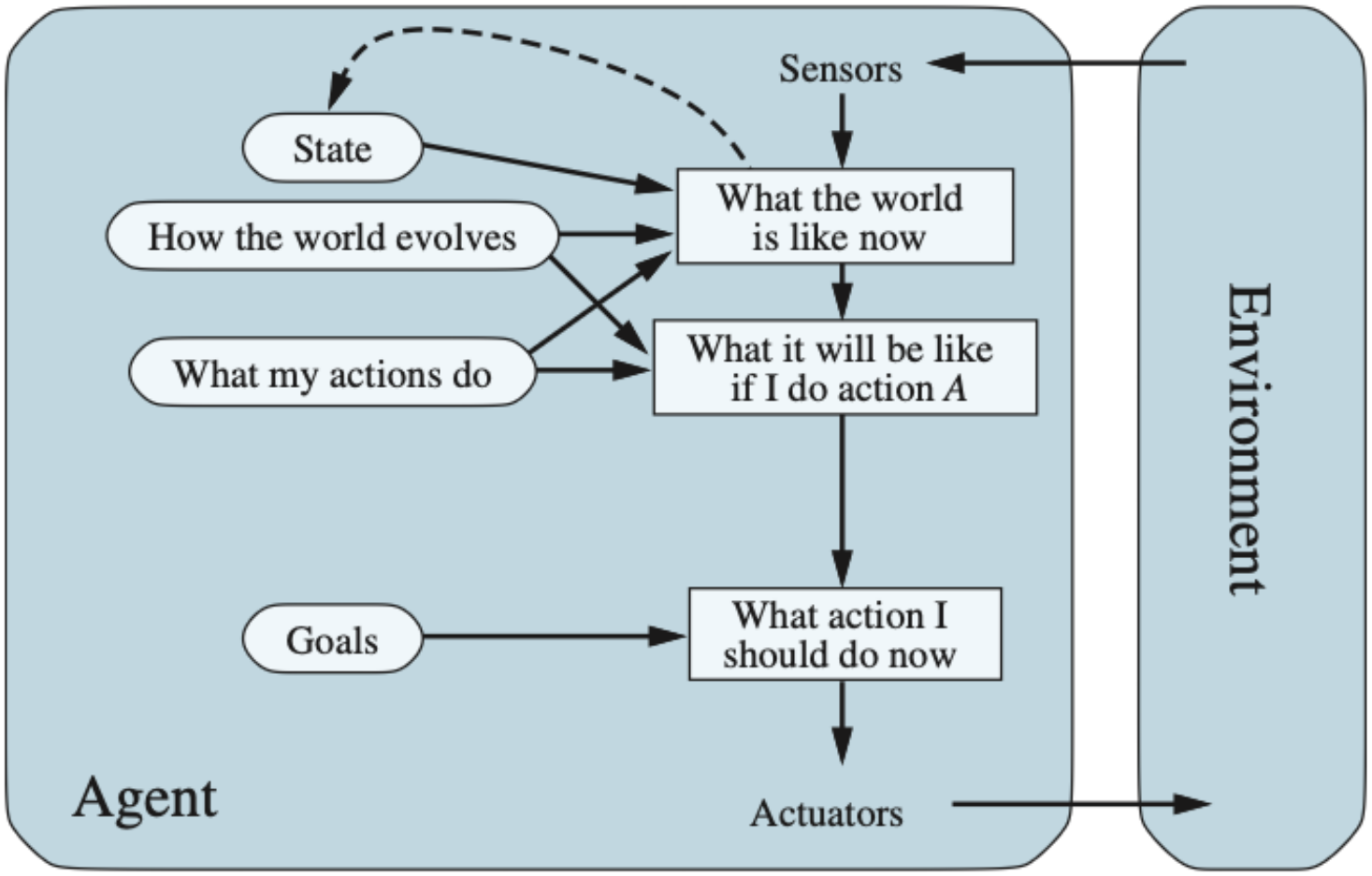
```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              transition_model, a description of how the next state depends on
                    the current state and action
              sensor_model, a description of how the current world state is reflected
                    in the agent's percepts
              rules, a set of condition–action rules
              action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, transition_model, sensor_model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Goal-Based Agents 1/2

- Knowing something about the current state of the environment is not always enough to decide what to do.
- At a road junction, the taxi can turn left, turn right, or go straight. The correct decision depends on where the taxi is trying to get to.
- The agent needs goal information that describes situations that are desirable—for example, being at a particular destination.

Goal-Based Agents 2/2

- The agent program can combine this goal information with the model to choose actions that achieve the goal.
- Sometimes goal-based action selection is straightforward. Sometimes the agent has to consider long sequences of actions to find a way to achieve the goal. This is the domain of search and planning.


Sensors
State
What the world is like now
How the world evolves
What my actions do
What it will be like if I do action A
Goals
What action I should do now
Agent
Actuators
Environment

Advantages of Goal-Based Agents 1/2

- Although goal-based agents may appear less efficient than reflex agents, they are more flexible.
- The knowledge that supports their decisions is represented explicitly and can be modified.
- A goal-based agent's behavior can easily be changed to go to a different destination, simply by specifying that destination as the goal.

Advantages of Goal-Based Agents 2/2

- A reflex agent's rules for when to turn and when to go straight work only for a single destination; they must all be replaced to go somewhere new.
- This flexibility makes goal-based agents more adaptable and suitable for complex environments with multiple possible objectives.

Utility-Based Agents

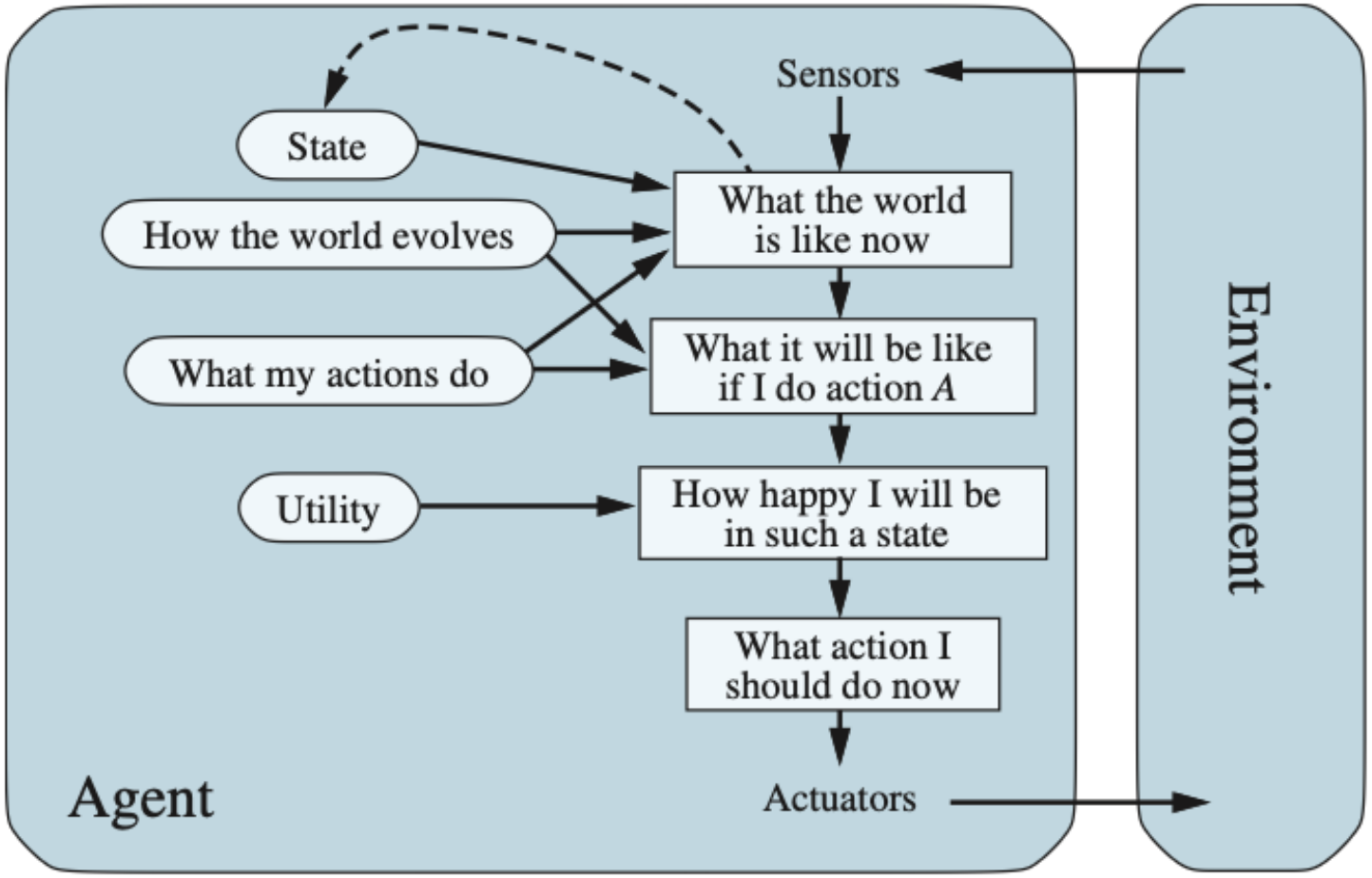
- Goals alone are not enough to generate high-quality behavior in most environments.
- Many action sequences will get the taxi to its destination, but some are quicker, safer, more reliable, or cheaper than others.
- Goals just provide a crude binary distinction between "happy" and "unhappy" states.
- Utility provides a way to compare different world states according to exactly how happy they would make the agent.
- An agent's utility function is essentially an internalization of the performance measure.

Utility Functions and Expected Utility 1/2

- When there are conflicting goals, only some of which can be achieved, the utility function specifies the appropriate tradeoff.
- When there are several goals that the agent can aim for, none of which can be achieved with certainty, utility provides a way in which the likelihood of success can be weighed against the importance of the goals.
- Technically speaking, a rational utility-based agent chooses the action that maximizes the expected utility of the action outcomes.

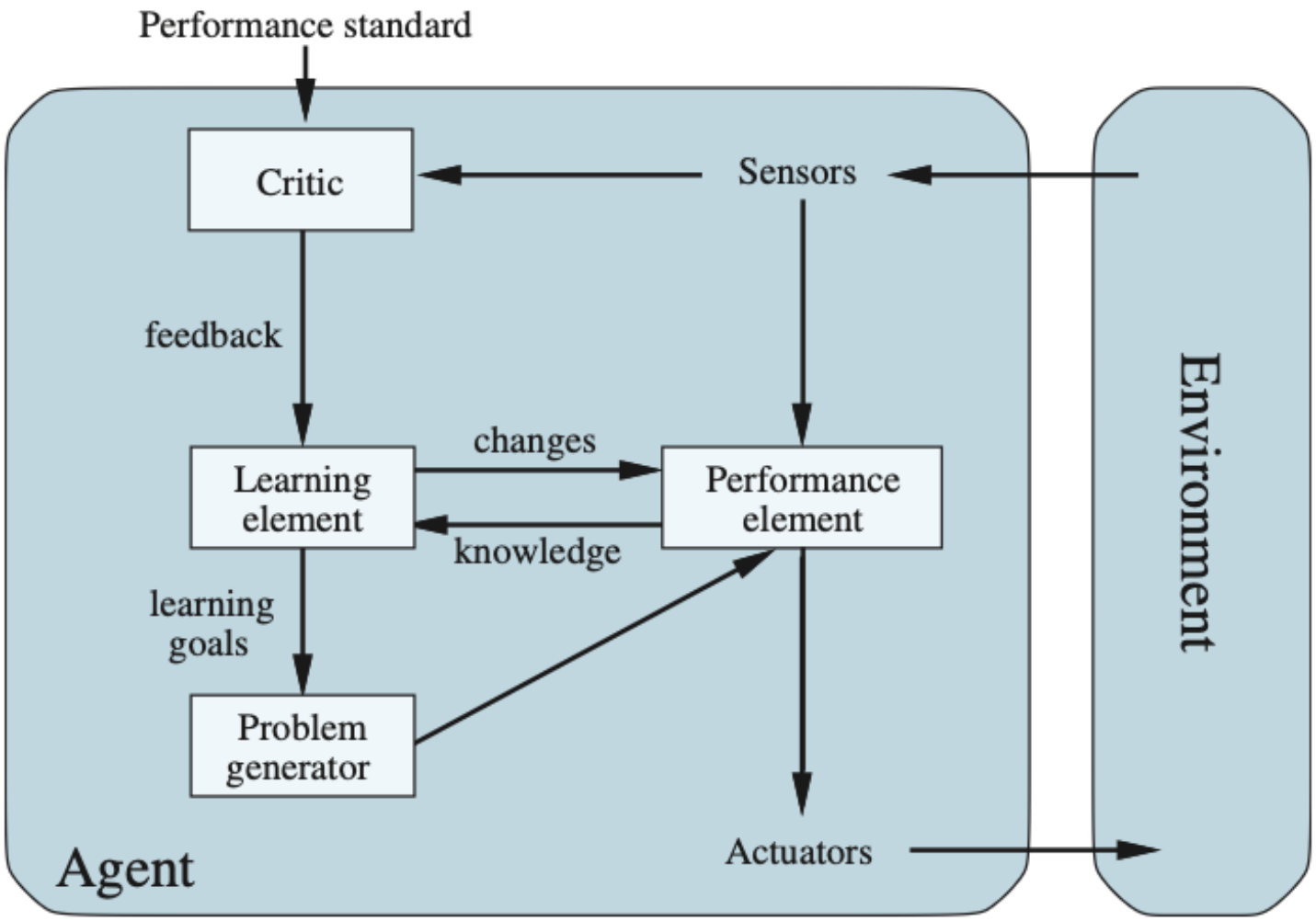
Utility Functions and Expected Utility 2/2

- Expected utility is the utility the agent expects to derive, on average, given the probabilities and utilities of each outcome.
- A utility-based agent has to model and keep track of its environment, tasks that have involved a great deal of research on perception, representation, reasoning, and learning.


Sensors
State
What the world is like now
How the world evolves
What my actions do
What it will be like if I do action A
Utility
How happy I will be in such a state
What action I should do now
Agent
Actuators
Environment

Learning Agents

- We have described agent programs with various methods for selecting actions, but we have not explained how the agent programs come into being.
- Turing proposed building learning machines and then teaching them, rather than programming them by hand.
- In many areas of AI, this is now the preferred method for creating state-of-the-art systems.
- Any type of agent (model-based, goal-based, utility-based, etc.) can be built as a learning agent.
- Learning allows the agent to operate in initially unknown environments and to become more competent than its initial knowledge alone might allow.

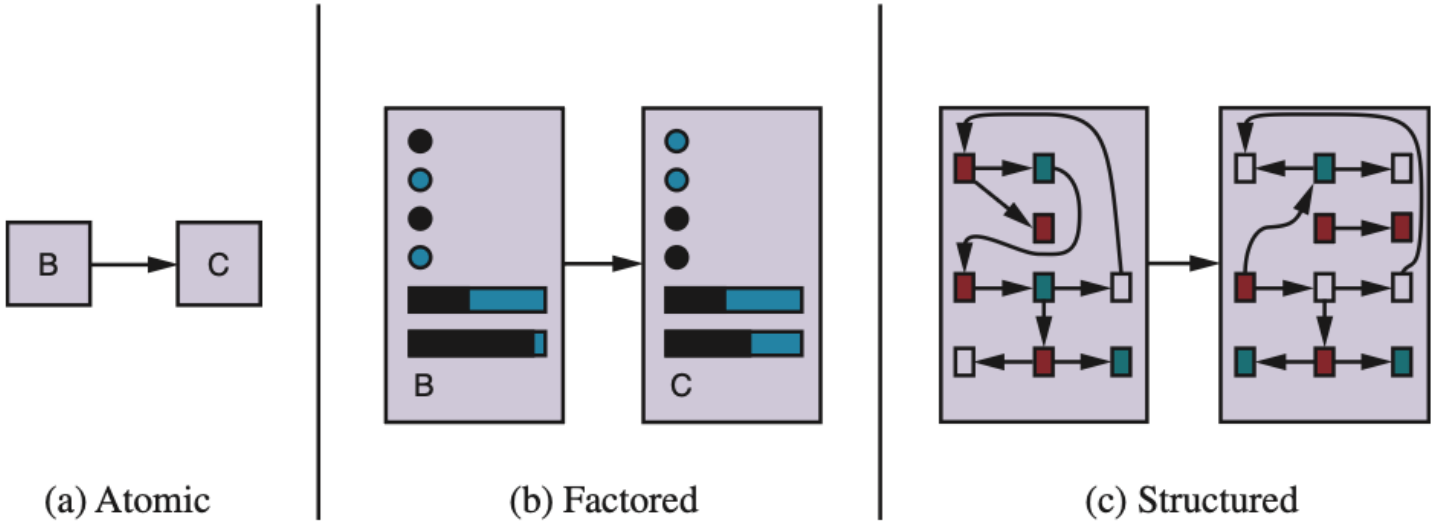

Performance standard
Critic
Sensors
feedback
changes
Learning element
Performance element
knowledge
learning goals
Problem generator
Actuators
Agent
Environment

Learning Agent Components

- The learning element is responsible for making improvements.
- The performance element is responsible for selecting external actions; it is what we have previously considered to be the entire agent.
- The critic tells the learning element how well the agent is doing with respect to a fixed performance standard.
- The problem generator is responsible for suggesting actions that will lead to new and informative experiences.
- Learning can modify any of the "knowledge" components: condition-action rules, reflex components, utility functions, transition models, and sensor models.

Representations of Agent States

- Agent programs consist of various components that answer questions such as "What is the world like now?" and "What action should I do now?"
- The components can represent the environment along an axis of increasing complexity and expressive power: atomic, factored, and structured.
- In an atomic representation, each state of the world is indivisible and has no internal structure.
- In a factored representation, each state is split into a fixed set of variables or attributes, each of which can have a value.
- In a structured representation, objects such as cows and trucks and their various relationships can be described explicitly.



(a) Atomic (b) Factored (c) Structured

Localist vs. Distributed Representations 1/2

- Another axis for representation involves the mapping of concepts to locations in physical memory.
- If there is a one-to-one mapping between concepts and memory locations, we call that a localist representation.
- If the representation of a concept is spread over many memory locations, and each memory location is employed as part of the representation of multiple different concepts, we call that a distributed representation.

Localist vs. Distributed Representations 2/2

- Distributed representations are more robust against noise and information loss.
- In a distributed representation, you can think of each concept representing a point in multidimensional space, and if you garble a few bits you move to a nearby point in that space, which will have similar meaning.

Summary: Key Concepts 1/2

- An agent is something that perceives and acts in an environment.
- A rational agent acts so as to maximize expected performance, given the percept sequence and built-in knowledge.
- A task environment specification includes performance measure, environment, actuators, and sensors (PEAS).

Summary: Key Concepts 2/2

- Task environments vary along several dimensions: observable/partially observable, single/multi-agent, deterministic/nondeterministic, episodic/sequential, static/dynamic, discrete/continuous, known/unknown.

- Agent designs include simple reflex, model-based reflex, goal-based, and utility-based agents, all of which can be enhanced with learning capabilities.

Conclusions

- The concept of rational agents provides a unified framework for understanding and designing AI systems.
- The structure of an agent program must match the characteristics of its task environment.
- Simple reflex agents work for fully observable environments but struggle with partial observability.
- Model-based agents can handle partial observability by maintaining internal state.
- Goal-based and utility-based agents can plan ahead and balance competing objectives.
- Learning enables agents to improve their performance over time and adapt to unknown environments.